RAPPORT DE PROJET C++

Pricing d'Options par Résolution d'EDP
Modèle de Black-Scholes & Schéma de Crank-Nicolson

Auteurs :
Yléo AUBRY
Yepi Sylvian Jean-Mederick BESSI
Ayman GRAZIANA

Date de rendu :
16 décembre 2025



Résumé

Ce projet implémente un moteur de valorisation d'options (Pricer) en C++ basé sur la résolution numérique de l'équation aux dérivées partielles de Black-Scholes. L'architecture utilise une transformation logarithmique du sous-jacent pour garantir la stabilité des coefficients, couplée à un schéma de différences finies implicite (Crank-Nicolson) résolu par l'algorithme de Thomas. Le programme calcule le prix ainsi que les sensibilités (Grecques) avec une précision de l'ordre de 10^{-3}.

Table des matières

1 Cadre Mathématique et Numérique **3**
1.1 L'Équation Initiale et ses Limites . . . 3
1.2 Transformation Logarithmique (Heat Equation) . . . 3
1.3 Discrétisation par Crank-Nicolson . . . 3
1.4 Résolution Algorithmique . . . 4
1.5 Calcul des Grecques . . . 4

2 Architecture Logicielle et Organisation **5**
2.1 Arborescence du Projet . . . 5
2.2 Description des Modules . . . 5
2.2.1 La Couche Mathématique (`LinearSolver`) . . . 5
2.2.2 La Couche Financière "Produit" (`Payoff`) . . . 5
2.2.3 La Couche Moteur (`PDESolver`) . . . 6
2.3 Flux de Données . . . 6

3 Manuel d'Utilisation et Guide de Compilation **7**
3.1 Pré-requis Système . . . 7
3.2 Procédure de Compilation (Recommandée) . . . 7
3.3 Procédure de Compilation Manuelle (Alternative) . . . 7
3.4 Exécution et Fonctionnement . . . 7
3.4.1 Mode 1 : Suite de Tests (Benchmark) . . . 8
3.4.2 Mode 2 : Mode Utilisateur . . . 8

1 Cadre Mathématique et Numérique

L'objectif du programme est de déterminer la valeur $V(S,t)$ d'une option européenne en résolvant l'Equation aux Dérivées Partielles (EDP) de Black-Scholes-Merton.

1.1 L'Équation Initiale et ses Limites

Sous les hypothèses standard (marché sans arbitrage, volatilité constante, taux sans risque constant), le prix de l'option vérifie :

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{1}$$

Avec la condition terminale $V(S,T) = \text{Payoff}(S)$.

Problème numérique : La discrétisation directe de l'équation (1) est problématique car les coefficients de diffusion ($\frac{1}{2}\sigma^2 S^2$) et de convection (rS) dépendent de la variable d'espace S. Cela entraîne deux conséquences néfastes pour le code :

1. La matrice du système linéaire doit être recalculée à chaque ligne de la grille, ce qui est coûteux.
2. La stabilité du schéma numérique est difficile à garantir pour des S grands (terme en S^2 explosif).

1.2 Transformation Logarithmique (Heat Equation)

Pour pallier ces problèmes, nous avons implémenté un changement de variable spatial. Posons $x = \ln(S)$. Le domaine $S \in [0, +\infty[$ devient $x \in]-\infty, +\infty[$. En appliquant la règle de la chaîne (Chain Rule) :

$$\frac{\partial V}{\partial S} = \frac{1}{S}\frac{\partial V}{\partial x}, \quad \frac{\partial^2 V}{\partial S^2} = \frac{1}{S^2}\left(\frac{\partial^2 V}{\partial x^2} - \frac{\partial V}{\partial x}\right)$$

En substituant dans (1), les termes en S et S^2 s'annulent parfaitement. L'équation devient une convection-diffusion à **coefficients constants** :

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 \frac{\partial^2 V}{\partial x^2} + \left(r - \frac{1}{2}\sigma^2\right)\frac{\partial V}{\partial x} - rV = 0 \tag{2}$$

C'est cette équation précise que notre classe `PDESolver` résout. L'avantage majeur est que la matrice de résolution est constante, permettant de la pré-calculer une seule fois (méthode `precomputeMatrices`).

1.3 Discrétisation par Crank-Nicolson

Nous utilisons une grille spatio-temporelle finie.
— Espace : N points x_i uniformément répartis entre $\ln(S_{\min})$ et $\ln(S_{\max})$.
— Temps : M pas de temps Δt.

Le schéma de Crank-Nicolson est une moyenne arithmétique entre le schéma Explicite (instable) et le schéma Implicite (stable mais diffusif). Il offre une précision en $O(\Delta t^2, \Delta x^2)$ et une stabilité inconditionnelle.

L'équation discrétisée s'écrit :

$$\frac{V_i^{n+1} - V_i^n}{\Delta t} = \frac{1}{2}\mathcal{L}_h V_i^{n+1} + \frac{1}{2}\mathcal{L}_h V_i^n \tag{3}$$

Où $\mathcal{L}_h$ est l'opérateur discret spatial. Cela mène au système linéaire matriciel :

$$\mathbf{A}V^n = \mathbf{B}V^{n+1} + \text{Limites} \tag{4}$$

Dans notre code, $\mathbf{A}$ (matrice implicite à gauche) et $\mathbf{B}$ (matrice explicite à droite) sont tridiagonales.

1.4 Résolution Algorithmique

À chaque pas de temps (en remontant de T à 0), nous devons résoudre le système $\mathbf{A}X = D$. Plutôt que d'utiliser une inversion matricielle générique coûteuse en $O(N^3)$, nous avons implémenté l'**Algorithme de Thomas** (TDMA) dans le module `LinearSolver`. Cet algorithme exploite la structure tridiagonale pour trouver la solution exacte en complexité linéaire $O(N)$, garantissant une exécution quasi-instantanée même pour des grilles fines ($N = 1000$).

1.5 Calcul des Grecques

Les sensibilités sont extraites directement de la grille finale par différences finies, en tenant compte du changement de variable :

— **Delta (Δ)** : $\frac{\partial V}{\partial S} = \frac{1}{S}\frac{\partial V}{\partial x} \approx \frac{V_{i+1} - V_i}{S_i \Delta x}$

— **Gamma (Γ)** : $\frac{\partial^2 V}{\partial S^2} = \frac{1}{S^2}\left(\frac{\partial^2 V}{\partial x^2} - \frac{\partial V}{\partial x}\right)$

2 Architecture Logicielle et Organisation

Afin de garantir la maintenabilité et l'extensibilité du projet, nous avons adopté une architecture modulaire respectant les standards industriels du C++ (séparation Interface/Implémentation) et les principes de conception SOLID.

2.1 Arborescence du Projet

L'organisation des fichiers suit la convention standard de CMake, séparant clairement les déclarations (`.h`) des définitions (`.cpp`) :

Listing 1 – Structure des fichiers du projet

```
Projet_EDP/
|-- CMakeLists.txt          # Script de compilation automatise
|-- include/                # Interfaces (Contrats)
|   |-- LinearSolver.h      # Declaration du solveur matriciel
|   |-- Payoff.h            # Classe abstraite des produits financiers
|   |-- PDESolver.h         # Moteur de resolution EDP
|   |-- Interface.h         # Gestion des entrees/sorties utilisateur
|-- src/                    # Implementations (Code source)
    |-- LinearSolver.cpp    # Algorithme de Thomas
    |-- PDESolver.cpp       # Logique de Crank-Nicolson
    |-- Interface.cpp       # Code de l'interface console
    |-- main.cpp            # Point d'entree et tests unitaires
```

2.2 Description des Modules

Le code est divisé en trois couches distinctes de responsabilité :

2.2.1 La Couche Mathématique (`LinearSolver`)

Ce module est totalement indépendant de la finance. Il fournit les outils d'algèbre linéaire nécessaires à la résolution numérique.

- **Rôle :** Résoudre des systèmes linéaires tridiagonaux $Ax = d$.
- **Algorithme :** Thomas (TDMA), choisi pour sa complexité en $O(N)$ contre $O(N^3)$ pour une inversion standard.
- **Optimisation :** Utilisation de références constantes (`const std::vector&`) pour éviter la copie coûteuse de vecteurs en mémoire.

2.2.2 La Couche Financière "Produit" (`Payoff`)

Cette couche définit *ce que* l'on price. Nous avons utilisé le **Design Pattern Strategy** via le polymorphisme.

- **Classe Abstraite :** `Payoff` définit une méthode virtuelle pure `operator()(double spot)`.
- **Flexibilité :** Le moteur de pricing ne connaît pas le type d'option (Call, Put, Digitale). Il manipule une référence générique `Payoff&`. Cela permet d'ajouter de nouveaux types d'options sans modifier une seule ligne du moteur de calcul (Principe Open/Closed).

2.2.3 La Couche Moteur (`PDESolver`)

C'est le cœur du programme qui orchestre la résolution.

— **Initialisation :** Conversion des paramètres de marché (S, K, r, σ) en paramètres de grille logarithmique.

— **Pré-calcul :** Les matrices de l'équation de la chaleur sont constantes. Elles sont construites une seule fois lors de l'instanciation pour optimiser la boucle temporelle.

— **Résolution :** Implémente la boucle de temps *backward* et gère les conditions aux limites de Dirichlet spécifiques au type d'option (déduites asymptotiquement).

2.3 Flux de Données

Le programme suit un flux d'exécution séquentiel :

1. L'**Interface** récupère les paramètres utilisateur et instancie un objet `Payoff` concret (ex : `PayoffCall`).
2. Le **PDESolver** est initialisé avec les paramètres de grille.
3. La méthode `solve()` est appelée. Elle construit le maillage, et à chaque pas de temps, délègue la résolution du système linéaire au `LinearSolver`.
4. Les résultats (Prix, Delta, Gamma) sont renvoyés sous forme d'une structure `PricingResults` pour être affichés ou exportés.

3 Manuel d'Utilisation et Guide de Compilation

Ce chapitre détaille la procédure exacte pour compiler le code source et exécuter le programme de pricing. Le projet a été conçu pour être portable et fonctionne sous Windows (via MinGW ou WSL), macOS et Linux.

3.1 Pré-requis Système

Avant de compiler, assurez-vous de disposer des outils suivants :
— **Compilateur C++ :** Compatible C++17 (ex : `g++`, `clang++` ou MSVC).
— **Système de Build :** `CMake` (version 3.10 ou supérieure).
— **Terminal :** PowerShell, Bash ou Terminal VS Code.

3.2 Procédure de Compilation (Recommandée)

Nous utilisons **CMake** pour automatiser la génération des fichiers de construction. Cette méthode, dite "Out-of-Source", permet de ne pas polluer le code source avec des fichiers temporaires.

1. Ouvrez un terminal à la racine du dossier `Projet_EDP`.
2. Créez un répertoire de construction et déplacez-vous dedans :

   ```
   mkdir build
   cd build
   ```

3. Générez les fichiers de makefile (CMake détectera automatiquement les sources dans `src/` et les headers dans `include/`) :

   ```
   cmake ..
   ```

   Note : Sous Windows avec MinGW, si la commande échoue, utilisez : `cmake -G "Unix Makefiles" ..`
4. Compilez l'exécutable :

   ```
   make
   ```

   Si la compilation réussit, un fichier exécutable `SolverEDP` (ou `SolverEDP.exe`) est généré dans le dossier `build`.

3.3 Procédure de Compilation Manuelle (Alternative)

Si CMake n'est pas disponible, il est possible de compiler directement avec `g++` en ligne de commande. Attention à bien lier tous les fichiers sources et à inclure le dossier des headers.

Depuis la racine du projet :

```
g++ src/main.cpp src/LinearSolver.cpp src/PDESolver.cpp src/Interface.cpp -
```

3.4 Exécution et Fonctionnement

Pour lancer le programme, exécutez le binaire généré :

```
./SolverEDP
```

Le programme propose alors un menu interactif à deux options :

3.4.1 Mode 1 : Suite de Tests (Benchmark)

Ce mode lance une batterie de tests unitaires pour valider l'intégrité du moteur :
— Validation de l'inversion matricielle (Algo de Thomas).
— Validation de la logique des Payoffs (Call/Put).
— **Test de Pricing :** Compare le résultat de l'EDP avec la formule fermée de Black-Scholes pour un set de paramètres fixes. L'erreur absolue et les Grecques sont affichées.

3.4.2 Mode 2 : Mode Utilisateur

Ce mode permet de pricer une option personnalisée. L'utilisateur est invité à saisir :
— Le prix du sous-jacent (S_0) et le Strike (K).
— La maturité (T) en années et le taux sans risque (r).
— La volatilité (σ) et la finesse de la grille (N, M).
Le programme calcule ensuite le prix, le Delta et le Gamma, et les affiche dans la console.